

Chapter 12

Building a RAG System — Part 2

A hands-on walkthrough of building a complete "Chat with any YouTube Video" RAG application using LangChain — from transcript ingestion to a fully chained, production-flavoured pipeline.

WHAT THIS CHAPTER COVERS

- ▶ YouTube Chat problem statement & plan of action
- ▶ Fetching transcripts via YouTubeTranscriptApi
- ▶ Text splitting with RecursiveCharacterTextSplitter
- ▶ Embedding generation & FAISS vector store
- ▶ Building a similarity-based Retriever
- ▶ Augmentation: merging context + question into a prompt
- ▶ Generation using a Chat LLM
- ▶ Chaining everything with RunnableParallel
- ▶ Upgrading a RAG system to industry grade
- ▶ RAG evaluation, advanced retrieval & system design variants

Building a RAG System — Part 2

RECAP — CHAPTER 11 (RETRIEVERS)

Chapter 11 covered Retrievers as query-in/Document-out Runnables: Wikipedia Retriever, Vector Store Retriever, MMR, Multi Query Retriever, and Contextual Compression Retriever. This chapter puts a Vector Store Retriever to work inside a complete, end-to-end RAG application, and then closes the RAG mini-series by discussing how such a system is upgraded to industry grade.

1. Problem Statement — YouTube Chat

This chapter is the practical continuation of the previous (theory-focused) RAG lecture. The goal is to build a real, working RAG-based system end-to-end using LangChain, reusing everything learned so far about Document Loaders, Text Splitters, and Vector Stores.

Definition

YouTube Chat is the chosen problem statement: a RAG-based system that lets a user chat in real time with the content of any YouTube video, without watching the whole video.

Why It Exists

Many YouTube videos — especially podcasts — run for two to three hours. If a viewer only needs to know whether a specific topic was discussed, or wants a quick summary, watching the entire video is wasteful. A chat-based system that already "knows" the transcript can answer such questions instantly.

How It Works (User Perspective)

1. The user is watching a long video (e.g. a 2-3 hour podcast).
2. A specific question comes to mind, e.g. "Does this podcast talk about AI?" or "Summarize this video in five bullet points."
3. The question is typed into the YouTube Chat system.
4. The system instantly replies with a grounded answer, based only on that video's transcript.

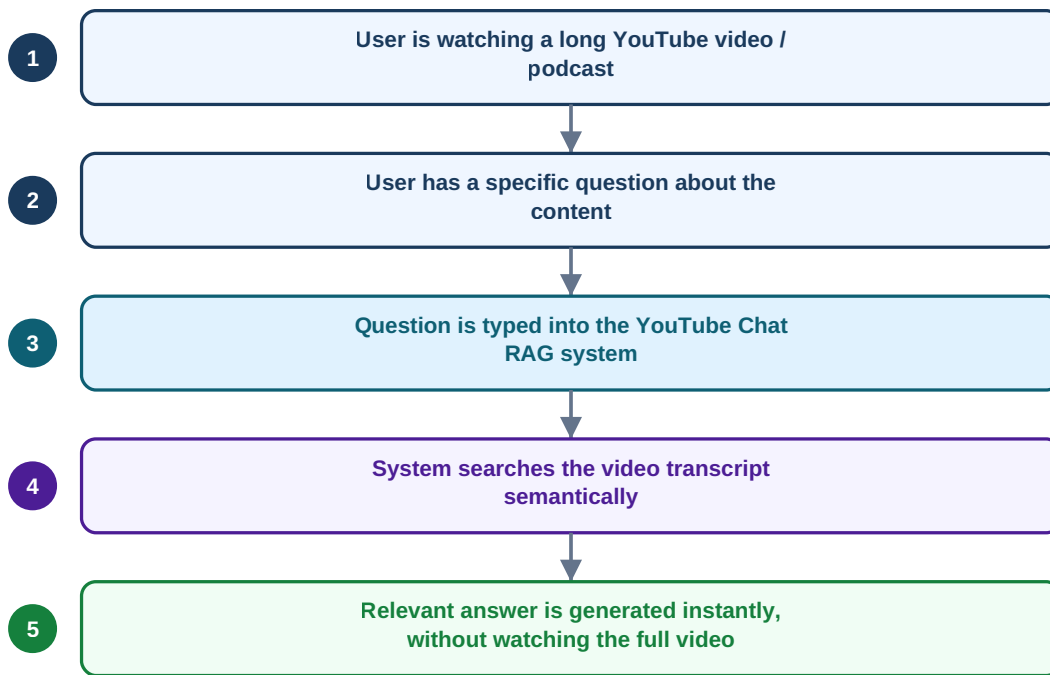


Figure 12.1 — How a user interacts with the YouTube Chat RAG system.

Possible Final Product Forms

Option	Description	Skill Required
Chrome Plugin	User installs a plugin; a chat interface opens directly on top of the YouTube video while it plays.	HTML, CSS, JavaScript
Streamlit Website	User pastes the video URL into a website; a chat window opens for that video.	Streamlit (Python-only)

Note: This chapter focuses entirely on the RAG functionality, not the UI. The full implementation is built inside a Google Colab notebook. Building a proper UI (Streamlit app or Chrome plugin) around this working backend is recommended as a follow-up exercise.

🔄 Quick Revision — Problem Statement

- YouTube Chat = a RAG system that lets users chat with any YouTube video's content in real time.
- Solves the problem of not wanting to watch very long videos just to check one detail.
- Final product could be a Chrome plugin (needs HTML/CSS/JS) or a Streamlit website (Python-only).
- This chapter builds only the RAG functionality inside a Google Colab notebook; UI is a suggested next step.

2. Plan of Action

The project follows the exact same four-stage RAG architecture discussed in the previous (theory) chapter: **Indexing** → **Retrieval** → **Augmentation** → **Generation**.



Figure 12.2 — The four canonical RAG stages used to build this project.

1. INDEXING

Fetch the transcript of the target YouTube video, split it into chunks, generate embeddings for each chunk, and store them in a vector store.

2. RETRIEVAL

Build a simple similarity-search-based retriever. Sending it a query returns the most relevant chunks from the vector store.

3. AUGMENTATION

Merge the retrieved chunks (context) and the original query into a single, well-structured prompt.

4. GENERATION

Send the augmented prompt to an LLM, which understands the query and the context, and produces the final response.

Important

Two ways exist to fetch a YouTube transcript: LangChain's built-in **YouTube Loader**, or YouTube's own transcript API used directly. This chapter uses the direct API approach because the LangChain YouTube Loader was found to be buggy — it worked for some videos but failed (code errors) for others.

🔄 Quick Revision — Plan of Action

- The project reuses the standard 4-stage RAG architecture: Indexing, Retrieval, Augmentation, Generation.
- Indexing = load transcript, split into chunks, embed, store in a vector database.
- Retrieval = similarity search against the vector store using the user's query.
- Augmentation = merge retrieved context + question into one prompt.
- Generation = LLM produces the final grounded answer.
- The direct YouTube Transcript API is used instead of LangChain's YouTube Loader, because the loader was unreliable/buggy for some videos.
- The full pipeline is first built step by step, then converted into a single LangChain chain at the end.

3. Step 1 — Indexing: Document Ingestion

Definition

Document ingestion is the process of fetching the raw transcript text of a target YouTube video and bringing it into the code as a single string, ready for downstream processing.

How It Works

1. Extract only the **video ID** from the YouTube URL (not the full URL).
2. Call `YouTubeTranscriptApi`'s `fetch` method with the video ID and the desired transcript language.
3. The raw result is a list of dictionaries, each representing one subtitle line with its **text**, **start timestamp**, and **duration** on screen.
4. Loop over this list and join all the text fields together into one large string — the complete transcript of the video.
5. Wrap the `fetch` call in a `try/except` block to catch the case where the video has no captions available (`TranscriptsDisabled`).

```
from youtube_transcript_api import YouTubeTranscriptApi, TranscriptsDisabled

# Only the video ID is needed, not the full URL
video_id = "Gfr50f6ZBvo"

try:
    # If language isn't specified, the API returns the best available match
    ytt_api = YouTubeTranscriptApi()
    transcript_list = ytt_api.fetch(video_id, languages = ["en"])

    # Convert the list of timestamped dicts into one plain-text transcript
    transcript = " ".join(line.text for line in transcript_list)

except TranscriptsDisabled:
    print("No captions available for this video!")
```

Each entry fetched from the API looks conceptually like a dictionary with the currently displayed subtitle text, the timestamp (in seconds) at which it appears, and how long it stays on screen before the next line loads. The join step discards the timing metadata and produces one continuous block of text representing the entire spoken content of the video.

Common Mistake

Pasting the full YouTube URL instead of just the video ID will break the `fetch` call. Always extract only the ID portion from the URL.

Common Mistake

Requesting `languages=["en"]` for a video whose captions are only available in another language (e.g. Hindi) raises an error saying no English transcript exists. The fix is to request the correct language code (e.g. `"hi"`) matching the video's actual captions.

Quick Revision — Document Ingestion

- Use only the video ID (from the URL), never the full YouTube URL.
- YouTubeTranscriptApi.fetch() is used directly instead of LangChain's YouTube Loader, since the loader was unreliable.
- Raw transcript data arrives as a list of timestamped subtitle-line objects (text, start, duration).
- All text lines are joined into a single large string representing the full transcript.
- TranscriptsDisabled is caught to handle videos without captions.
- If a video's captions are in a non-English language, the requested language code must match (e.g. "hi" for Hindi).

4. Step 1 — Indexing: Text Splitting, Embedding & Vector Storage

B. Text Splitting

A full video transcript (especially for a 2-hour podcast) is far too large to use directly. It must be broken into smaller, manageable chunks.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size = 1000,
    chunk_overlap = 200
)

chunks = splitter.create_documents([transcript])
```

`RecursiveCharacterTextSplitter` is used with a chunk size of 1000 characters and an overlap of 200 characters. For a roughly two-hour podcast transcript, this produced **168 chunks**. Chunk size is a tunable hyperparameter — different values can be experimented with, but 1000/200 gave good results here. Individual chunks can be inspected by indexing into the resulting `chunks` list.

C & D. Embedding Generation and Storing in a Vector Store

Each chunk must be converted into a vector (embedding) and stored in a vector database so that semantic search can later be performed over them.

```
from langchain_nvidia_ai_endpoints import NVIDIAEmbeddings
from langchain_community.vectorstores import FAISS

# Embedding Model
embedding_model = NVIDIAEmbeddings(model = "nvidia/nemotron-3-embed-1b")

vector_store = FAISS.from_documents(chunks, embedding_model)
```

An `NVIDIAEmbeddings` model (`nvidia/nemotron-3-embed-1b`) is chosen as the embedding model, and **FAISS** is used as the vector store since it is simple and fast for a Colab-based project. After running `FAISS.from_documents`, every one of the 168 chunks receives a unique ID and is embedded into the store, ready to be searched. Any individual stored chunk can be looked up by its ID for inspection.

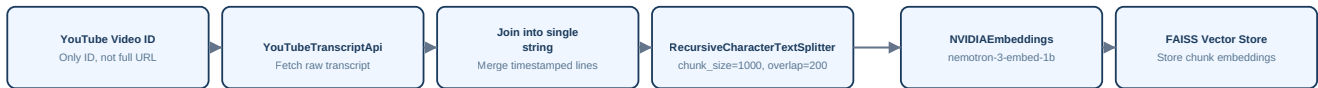


Figure 12.3 — The full indexing pipeline: video ID to a searchable FAISS vector store.

Best Practice

Treat chunk size and chunk overlap as hyperparameters to tune per use case rather than fixed constants. Larger overlaps preserve more context across chunk boundaries at the cost of more storage and redundant embeddings.

🔄 Quick Revision — Splitting, Embedding, Storage

- `RecursiveCharacterTextSplitter` with `chunk_size=1000`, `chunk_overlap=200` produced 168 chunks for a ~2 hour transcript.
- `NVIDIAEmbeddings` (`nvidia/nemotron-3-embed-1b`) is used as the embedding model.
- `FAISS.from_documents(chunks, embedding_model)` embeds and stores all chunks in one call.
- Every stored chunk gets a unique ID, retrievable individually for inspection.
- This completes the Indexing stage: Document Loading → Text Splitting → Embedding → Vector Storage.

5. Step 2 — Retrieval

Definition

Retrieval is the process of converting a retriever, built on top of the vector store, into an object that accepts a query and returns the most relevant Documents.

How It Works

1. Convert the vector store into a retriever using `.as_retriever()`.
2. Configure the **search type** as `"similarity"` and pass `search_kwargs={"k": 4}` to return the top 4 most similar chunks.
3. Since a retriever is itself a `Runnable`, call `.invoke(query)` on it to trigger a full retrieval cycle.
4. Internally, the retriever embeds the query into a vector, searches the vector store for the closest stored vectors, and returns the corresponding Document chunks.

```
retriever = vector_store.as_retriever(  
    search_type = "similarity",  
    search_kwargs = {"k": 4}  
)  
  
# Example query  
retriever.invoke("What is DeepMind?")
```

Running this with the query *"What is DeepMind?"* returned 4 Documents, since $k=4$ was configured, and the retriever judged all four as relevant to answering that question.

Important

A retriever always takes a **query (string)** as input and always returns a **list of Documents** as output. This input/output contract never changes regardless of which retrieval strategy is used internally.

Quick Revision — Retrieval

- `vector_store.as_retriever(search_type="similarity", search_kwargs={"k":4})` builds a simple similarity retriever.
- A retriever is a Runnable: calling `.invoke(query)` triggers the full search.
- Internally: embed the query → compare against stored vectors → return the closest Documents.
- Contract: input is always a query string, output is always a list of Documents.

6. Step 3 — Augmentation

Definition

Augmentation is the step where the retrieved context Documents and the original user query are merged together into a single, well-formed prompt that will be sent to the LLM.

How It Works

1. DEFINE THE LLM (OPTIONAL AT THIS STAGE, BUT CREATED HERE)

An LLM object can technically be created later, right before Generation, but it is convenient to instantiate it during this step.

2. DESIGN A PROMPT TEMPLATE

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate(
    template = """
    You are a helpful assistant.
    Answer ONLY from the provided transcript context.
    If the context is insufficient, just say you don't know.

    Context : {context}
    Question : {question}
    """,
    input_variables = ["context", "question"]
)
```

The prompt template explicitly instructs the model to answer **only** from the provided context and to admit when it doesn't know the answer, rather than hallucinating. It has two input variables: `context` and `question`.

3. RETRIEVE DOCUMENTS AND FORMAT THEM INTO A CONTEXT STRING

A prompt cannot be handed a list of Document objects directly — the page content of every retrieved Document must be concatenated into one string first.

```
# Retrieve documents for the question
question = "Is the topic of aliens discussed in this video? If yes then what was discussed?"
retrieved_docs = retriever.invoke(question)

# Concatenate the page_content of every retrieved document
context_text = "\n\n".join(doc.page_content for doc in retrieved_docs)
```

4. BUILD THE FINAL PROMPT

```
final_prompt = prompt.invoke({"context": context_text, "question": question})
```

The resulting `final_prompt` contains the system instructions, the concatenated context block, and the question — exactly what will be sent to the LLM.

Best Practice

Always instruct the model, inside the prompt, to answer strictly from the given context and to say "I don't know" when the context is insufficient. This is the simplest and most effective way to reduce hallucination in a RAG system.

Quick Revision — Augmentation

- PromptTemplate defines two input variables: context and question.
- Retrieved Documents cannot be passed directly to a prompt — their page_content must be joined into one string first.
- A custom format_docs-style function performs this concatenation.
- prompt.invoke({"context":..., "question":...}) produces the final prompt to send to the LLM.
- Explicitly instructing the model to answer only from context prevents hallucination.

7. Step 4 — Generation

Definition

Generation is the final step where the fully augmented prompt is sent to the LLM, which reads the question and the context and produces the final natural-language response.

How It Works

```
answer = model.invoke(final_prompt)
print(answer.content)
```

Calling `.invoke()` on the LLM with the final prompt returns a response object; the actual text is accessed via `.content`. Testing with the query about aliens correctly returned a grounded answer describing that the topic was discussed and summarizing the speaker's opinion, sourced only from the retrieved transcript context. A second test query about nuclear fusion similarly returned a correct, point-by-point answer grounded in the transcript.

Important

At this point the full RAG pipeline works: Indexing → Retrieval → Augmentation → Generation. However, each step so far has been invoked **manually and separately** — the retriever was invoked on its own, then the prompt was invoked on its own, then the model was invoked on its own. This is not clean or production-friendly.

Quick Revision — Generation

- `model.invoke(final_prompt)` sends the augmented prompt to the LLM.
- The generated text is accessed via the `.content` attribute of the response.
- At this point, all 4 RAG stages work correctly, but each is being invoked manually and separately.
- The next step is to convert this manual sequence into a single automated LangChain chain.

8. Converting the Pipeline into a Single Chain

Why It Exists

Manually invoking the retriever, then the prompt, then the LLM is error-prone and hard to maintain. LangChain's Runnable composition (LCEL) allows the entire pipeline to be collapsed into one chain, so that a single `.invoke()` call triggers the whole flow automatically — each component's output automatically becomes the next component's input.

How It Works

Looking back at the RAG architecture diagram, the full chain is really made of two connected parts:

1. A simple linear sub-chain: `prompt` → LLM → `parser`.
2. A parallel sub-chain that produces the two inputs the prompt needs: `context` and `question`.

The tricky part is that the prompt requires **two** inputs (`context` and `question`), but the chain as a whole only receives **one** input (the raw question string). `question` can be passed straight through unchanged, but `context` must first flow through the retriever and then through a formatting function — both running in parallel with the passthrough branch.

1. BUILD A FORMAT_DOCS HELPER FUNCTION

```
def format_docs(retrieved_docs):  
    return "\n\n".join(doc.page_content for doc in retrieved_docs)
```

This wraps the earlier manual concatenation logic into a reusable function.

2. BUILD THE PARALLEL CHAIN

```
from langchain_core.runnables import RunnableLambda, RunnableParallel, RunnablePassthrough  
  
parallel_chain = RunnableParallel(  
    {  
        "context" : retriever | RunnableLambda(format_docs),  
        "question" : RunnablePassthrough()  
    }  
)
```

`RunnableParallel` takes a dictionary of keys. Each key's value is itself a Runnable (or a chain of Runnables) that is executed on the **same input**, in parallel:

- **context** key: the input query flows through `retriever` (returns a list of Documents), then through `RunnableLambda(format_docs)` which wraps the plain Python function so it becomes a valid chain component, producing one large context string.
- **question** key: the input query flows through `RunnablePassthrough()`, which simply returns its input unchanged.

A plain Python function (like `format_docs`) cannot be used directly inside a LangChain chain unless it is itself a `Runnable` — wrapping it in `RunnableLambda` makes this possible.

```
parallel_chain.invoke("Who is Demis?")
```

Invoking the parallel chain alone returns a dictionary with two keys — `context` (the big concatenated string) and `question` (the original question), exactly matching what the prompt template expects.

3. BUILD THE MAIN CHAIN

```
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()

final_chain = parallel_chain | prompt | model | parser
```

The `parallel_chain`'s output dictionary (`context` + `question`) feeds directly into `prompt`; the `prompt`'s output feeds into `model`; the `model`'s output feeds into `parser`, which extracts the final string.

```
answer = final_chain.invoke("Can you summarize the video?")
print(answer)
```

A single `.invoke()` call now runs the entire Retrieval → Augmentation → Generation flow automatically.

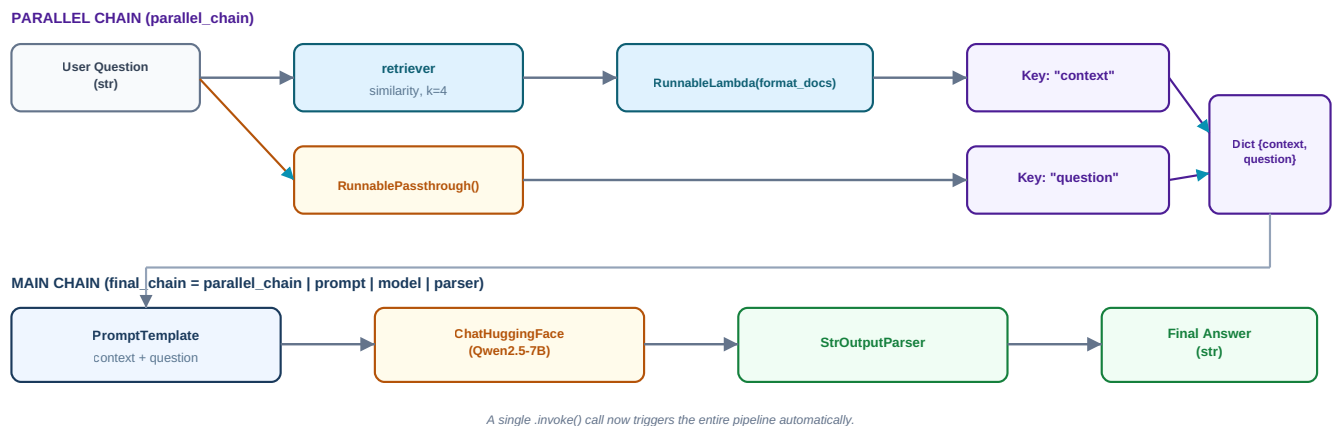


Figure 12.4 — Full chain architecture: `RunnableParallel` feeding a `prompt` → `model` → `parser` chain.

Interview Tip

Be ready to explain why `RunnableLambda` is required around `format_docs`: LangChain's pipe (`|`) composition only works between `Runnables`. A raw Python function isn't a `Runnable`, so it must be wrapped before it can participate in a chain.

Best Practice

Build and test each `Runnable` stage manually first (as done in Sections 3-7), and only combine them into a

single chain once every stage is verified to work correctly in isolation. This makes debugging far easier than writing the full chain from scratch.

Quick Revision — The Complete Chain

- The full RAG pipeline collapses into: `final_chain = parallel_chain | prompt | model | parser`.
- `RunnableParallel` runs multiple `Runnables` on the same input simultaneously, producing a dict of named outputs.
- `context key = retriever | RunnableLambda(format_docs)`; `question key = RunnablePassthrough()`.
- `RunnableLambda` wraps a plain Python function so it can be used inside a `LangChain` chain.
- `RunnablePassthrough()` simply returns its input unchanged — used here to forward the original question.
- The final chain needs only one `.invoke(question)` call to run Retrieval, Augmentation, and Generation automatically.

9. Upgrading a Simple RAG System to Industry Grade

The chain built above is a fully working, but basic, RAG system. Real, production-grade RAG systems used in the industry are considerably more complex and combine many additional techniques. The instructor organizes these improvements into distinct categories, giving a preview of what an *Advanced RAG* follow-up series will cover in depth.

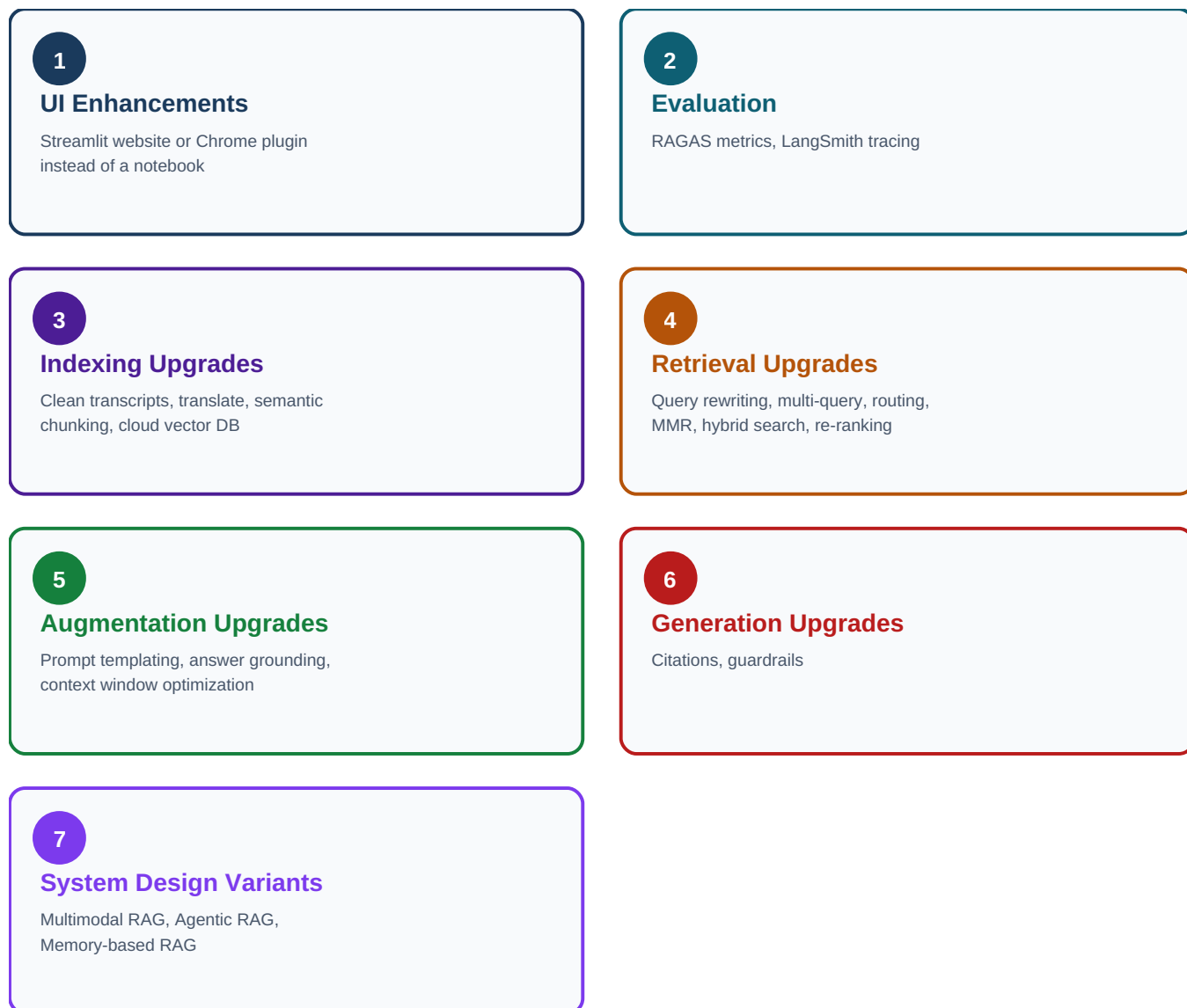


Figure 12.5 — Category map of improvements that turn a simple RAG system into an industry-grade one.

1. UI-Based Enhancements

Currently the system runs inside a Google Colab notebook, where the user must manually paste a video ID and run every cell before getting answers. A finished product would not work this way. Two upgrade paths were suggested:

- **Streamlit Website** — the user pastes a YouTube URL into a web page and chats with the video directly on that site.
- **Chrome Plugin** — the user installs a browser extension; opening any YouTube video activates the plugin, and a chat interface becomes available right there while watching.

2. Evaluation

The chapter so far only covered *building* a RAG system, never *evaluating* whether it performs correctly. Without evaluation, a system cannot be systematically improved. Industry-grade RAG systems are evaluated rigorously using dedicated tooling:

Tool / Metric	What It Measures
RAGAS (library)	A popular library purpose-built for evaluating RAG systems across multiple metrics.
Faithfulness	Whether the final generated answer is actually grounded in / related to the retrieved context.
Answer Relevancy	Whether the generated answer is properly relevant to the question that was asked.
Context Precision	How useful the retrieved context actually was for answering the question (i.e. how much of it was noise).
Context Recall	Whether all the useful information available in the vector store was successfully retrieved.
LangSmith (tool)	A tracing tool: attaches tracers to every step of the RAG pipeline to verify the whole pipeline is functioning correctly end-to-end.

3. Indexing Improvements

The ingestion, splitting, and storage stage of this simple system leaves significant room for improvement:

- **Transcript Cleanup** — auto-generated YouTube transcripts contain frequent errors; these should be cleaned up as part of ingestion.
- **Translation** — if a video's transcript comes in Hindi or another non-English language, it can be translated into English at the ingestion stage before further processing.
- **Semantic Chunking** — the plain RecursiveCharacterTextSplitter used here can split a chunk in the middle of a paragraph or idea; a Semantic Chunker (covered earlier in the Text Splitters chapter) avoids this by splitting along meaning boundaries.
- **Cloud-Based Vector Store** — FAISS is a very basic, local vector store. A real company-grade RAG system would instead use a cloud-based solution such as Pinecone.

4. Retrieval Improvements

Retrieval-time improvements are grouped into three phases:

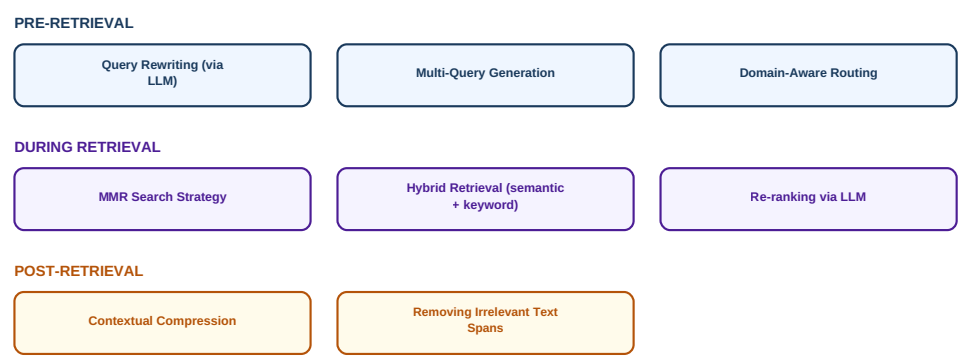


Figure 12.6 — Pre-retrieval, during-retrieval, and post-retrieval optimization techniques.

PRE-RETRIEVAL (BEFORE SEARCHING THE VECTOR STORE)

- **Query Rewriting** — the raw user query is often vague or not meaningful enough; an LLM can be placed in between to rewrite/improve the query before retrieval, improving retrieval quality.
- **Multi-Query Generation** — one vague query is expanded into multiple queries capturing different perspectives, then all results are merged (covered in the Retrievers chapter).
- **Domain-Aware Routing** — in complex systems with multiple retrievers set up, routing decides which retriever to trigger depending on the nature/domain of the incoming query.

DURING RETRIEVAL

- **MMR (Maximum Marginal Relevance)** — a search strategy that returns results that are both relevant and diverse from one another (also covered in the Retrievers chapter).
- **Hybrid Retrieval** — combining semantic (vector) search with traditional keyword search, then merging both result sets for the user.

POST-RETRIEVAL

- **Re-ranking** — rather than trusting the raw similarity-score ordering, an LLM creates a new ranking over all retrieved documents, improving what gets passed downstream.
- **Contextual Compression** — unnecessary, non-useful text within a retrieved document is stripped out, leaving only the meaningful part. This avoids wasting prompt space on irrelevant content (recall the "photosynthesis" example from the Retrievers chapter).

5. Augmentation Improvements

- **Prompt Templating** — carefully designing how the context and question are explained to the LLM, including giving it examples, so it understands precisely how to use the context to answer.
- **Answer Grounding** — explicitly instructing the LLM to answer strictly from the given context, never invent facts on its own, and never hallucinate.
- **Context Window Optimization** — LLMs can only process a limited number of tokens. If the retrieved context is too large, it may exceed this limit. The context is trimmed at prompt-design time so only the genuinely useful part remains, keeping the total prompt within the model's context window.

6. Generation Improvements

- **Answer with Citations** — instructing the LLM that whenever it answers, it should also indicate which part of the context the answer was drawn from.
- **Guardrailing** — preventing the LLM from producing inappropriate, incorrect, or harmful output. This is a whole practice area on its own and directly protects the end-user experience.

7. System Design Variants

Variant	Description
Multimodal RAG	The simple system built in this chapter only handles text (text in, text out). A multimodal RAG system can additionally process images and videos, not just text.
Agentic RAG	Instead of operating purely as a chatbot, the system operates as an AI agent. If answering a question requires an extra action (e.g. browsing the internet for additional information), the agent performs that action, merges the result with its existing context, and then answers.
Memory-Based RAG	A personalized RAG system that retains context across conversations over time — e.g. it recalls what was discussed a week earlier and factors that into the current answer.

Real World Insight

Everything covered across the RAG theory chapter and this practical chapter is only the surface of RAG. Building a genuine industry-grade RAG system surfaces many real problems, and an entire specialized field — *Advanced RAG* — has emerged in the industry to solve them. These advanced techniques are planned as a separate, dedicated playlist after this LangChain playlist concludes; they are intentionally out of scope for this playlist.

🔄 Quick Revision — Industry-Grade Upgrades

- **UI:** replace the Colab notebook with a Streamlit website or a Chrome plugin.
- **Evaluation:** use RAGAS (faithfulness, answer relevancy, context precision, context recall) and LangSmith (tracing).
- **Indexing:** clean transcripts, translate non-English transcripts, use semantic chunking, move to a cloud vector store like Pinecone.
- **Retrieval:** query rewriting, multi-query generation, and domain-aware routing (pre-retrieval); MMR and hybrid search (during retrieval); re-ranking and contextual compression (post-retrieval).
- **Augmentation:** better prompt templating, explicit answer grounding, and context window optimization.
- **Generation:** answers with citations, and guardrail against harmful/incorrect output.
- **System design variants:** Multimodal RAG, Agentic RAG, and Memory-Based RAG.
- A dedicated "Advanced RAG" playlist is planned to cover all these techniques in depth after the current LangChain playlist ends.

K. Key Takeaways

- **Problem Statement:** "YouTube Chat" — a RAG system that lets users chat in real time with any YouTube video's content, avoiding the need to watch long videos end-to-end.
- **Plan of Action:** The project follows the standard 4-stage RAG architecture — Indexing, Retrieval, Augmentation, Generation.

- **Document Ingestion:** `YouTubeTranscriptApi.fetch()` (used directly, not LangChain's buggy YouTube Loader) fetches a timestamped transcript, which is joined into one plain-text string.
- **Text Splitting:** `RecursiveCharacterTextSplitter` (`chunk_size=1000`, `chunk_overlap=200`) broke a 2-hour transcript into 168 chunks.
- **Embedding & Storage:** `NVIDIAEmbeddings` (`nemotron-3-embed-1b`) generated embeddings; FAISS stored them as a searchable vector store.
- **Retrieval:** `vector_store.as_retriever(search_type="similarity", search_kwargs={"k":4})` returns the top-4 most relevant Documents for any query.
- **Augmentation:** Retrieved Documents are concatenated via a `format_docs` function and merged with the question into a `PromptTemplate` with two input variables.
- **Generation:** `model.invoke(final_prompt)` produces the final grounded answer via `.content`.
- **Chaining:** `RunnableParallel` builds a `{{"context": retriever | RunnableLambda(format_docs), "question": RunnablePassthrough()}}` dict, piped into `prompt | model | parser` as `final_chain`, collapsing the whole pipeline into one `.invoke()` call.
- **Industry Upgrades:** A real RAG system additionally needs UI, evaluation (RAGAS, LangSmith), better indexing, advanced retrieval strategies, augmentation refinements, generation safeguards, and possibly a different system design (Multimodal, Agentic, or Memory-based RAG).
- **What's Next:** An "Advanced RAG" playlist is planned separately to cover all upgrade techniques in depth once the LangChain playlist concludes.

R. Revision Sheet

YouTube Chat (Problem Statement)

A RAG-based system enabling real-time chat with the content of any YouTube video, sparing the user from watching it fully.

YouTubeTranscriptApi

Direct API used to fetch a video's timestamped transcript by video ID; more reliable here than LangChain's built-in YouTube Loader.

RecursiveCharacterTextSplitter

Splits the large transcript string into overlapping chunks (`chunk_size=1000`, `chunk_overlap=200`) suitable for embedding.

NVIDIAEmbeddings + FAISS

Embedding model converts chunks to vectors; FAISS stores and indexes them for fast similarity search.

Retriever

A Runnable built via `vector_store.as_retriever()`; takes a query string, returns the top-k most similar Documents.

format_docs()

Custom function that concatenates the `page_content` of all retrieved Documents into a single context string.

PromptTemplate (context + question)

Instructs the LLM to answer only from the given context and to say "I don't know" when context is insufficient.

RunnableParallel

Runs multiple Runnables on the same input simultaneously, returning a dict of named outputs (here: context and question).

RunnableLambda

Wraps a plain Python function (like `format_docs`) so it becomes a valid Runnable usable inside a chain.

RunnablePassthrough

Returns its input unchanged; used to forward the original question straight into the prompt.

final_chain

parallel_chain | prompt | model | parser — the entire RAG pipeline collapsed into a single chain, triggered by one .invoke() call.

RAG Evaluation (RAGAS)

Measures faithfulness, answer relevancy, context precision, and context recall to judge RAG system quality.

LangSmith

A tracing tool used to verify every step of a RAG pipeline is functioning correctly end-to-end.

Advanced Retrieval Techniques

Query rewriting, multi-query generation, domain-aware routing, MMR, hybrid search, re-ranking, and contextual compression.

Augmentation Upgrades

Prompt templating, answer grounding (no hallucination), and context window optimization to fit token limits.

Generation Upgrades

Answer citations (showing source context) and guardrail (blocking harmful/incorrect output).

Multimodal / Agentic / Memory-Based RAG

System design variants: process images/video too, act as an autonomous agent with tools, or retain memory across conversations.

I. Interview Questions

Q1. Why was the direct YouTube Transcript API chosen over LangChain's built-in YouTube Loader for this project?

Q2. What are the input and output contracts of a LangChain Retriever, regardless of its internal search strategy?

Q3. Why can't retrieved Document objects be passed directly into a PromptTemplate, and how is this problem solved?

Q4. What is the purpose of `RunnableParallel`, and how is it used to produce the context and question keys needed by the prompt?

Q5. Why is `RunnableLambda` necessary when using a plain Python function like `format_docs` inside a LangChain chain?

Q6. What does `RunnablePassthrough` do, and why is it used for the question branch of the parallel chain?

Q7. Explain how prompt instructions like "answer only from the provided context" help reduce hallucination in a RAG system.

Q8. What are the four RAGAS evaluation metrics discussed, and what does each one measure?

Q9. Differentiate between pre-retrieval, during-retrieval, and post-retrieval optimization techniques, with an example of each.

Q10. What is the difference between Multimodal RAG, Agentic RAG, and Memory-Based RAG?

F. Further Reading

- [LangChain Documentation](#) — Runnable Interface (`RunnableParallel`, `RunnableLambda`, `RunnablePassthrough`)
- [LangChain Documentation](#) — LCEL (LangChain Expression Language)
- [FAISS Documentation](#) — Facebook AI Similarity Search
- [RAGAS Documentation](#) — RAG evaluation metrics and usage
- [LangSmith Documentation](#) — tracing and debugging LangChain pipelines
- [youtube-transcript-api \(PyPI\)](#) — official library reference
- [Pinecone Documentation](#) — cloud-based vector store as a FAISS alternative